

Assignment 0 Report

Student Name	Omar Moustafa
Student ID	900222400
Student Email	omoustafa@aucegypt.edu

Report template for Assignment1: **Linear Regression & Logistic Regression**
[https://github.com/sherifmost/DeepLearning/blob/master/Labs/lab1/ README.md](https://github.com/sherifmost/DeepLearning/blob/master/Labs/lab1/README.md)

General Instructions:

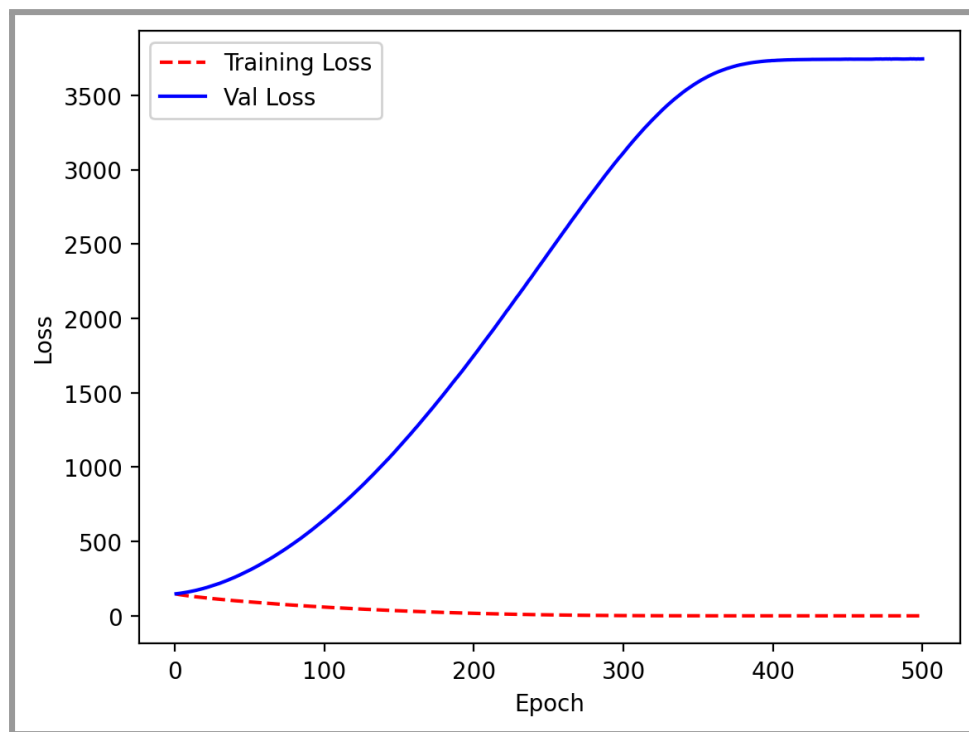
- Your work on the assignment and answers in the report are **individual effort**. Copying or any detected collaboration with a colleague will be severely penalized.
- In case you search for the answer to a question online (*only when explicitly mentioned that this is allowed*), **you must cite the reference or article used**.
- Do not modify the report template and only fill in the missing parts as indicated by the "TODO" keyword.
- The report template is self-explanatory and has the same flow as the requirements in the provided Google Collaboratory notebook, so make sure to follow the report while working on your experiments not to get stuck filling the report at the end. *The notebook will include references to what should be added to the report when necessary.*
- All the experiments and questions included in the report are **mandatory** and each have their own grade (**make sure you fill in all the required TODOs!**).
- In case you want to add in extra experiments or work done other than what is required, feel free to do so but make sure to indicate that this is extra work. For instance, in case you are provided with a table to put in the results of experimenting with different parameters and you want to include another parameter/ value other than those required, you can do so by adding extra rows to the table and highlighting them with a different color.
- Feel free to add any extra comments/conclusions/observations to any of the TODOs marked if you wanted.
- Your answers here regarding the experiments you did should be supported by plots generated using the Google Collaboratory notebooks, **otherwise will be considered plagiarism**.

- To submit the report, save it as a **PDF** file and upload it to the form provided in the instructions on the Blackboard. ***Make sure to submit the filled code notebooks as well, not just the report. (Failing to submit a notebook deduces 10 pts from the assignment grade).***

Part 1: Linear Regression (30 pts)

- Experimental Results (10 pts)

1. 1.1) The plot obtained when building and directly training the model (2.5 pts):

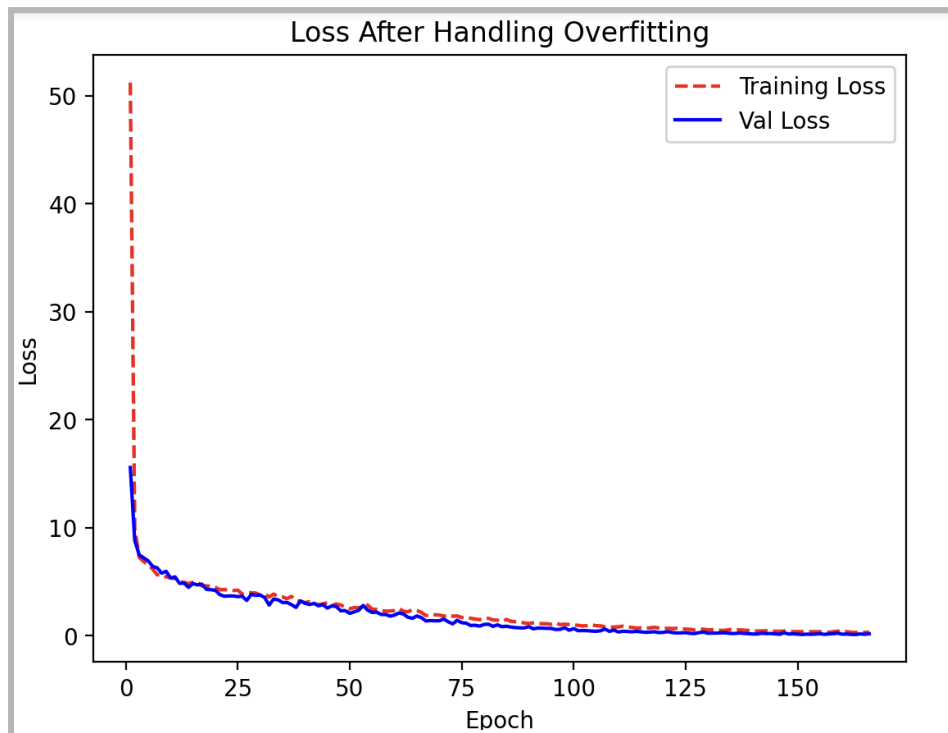


Question on 1.1: Does this plot show any overfitting or underfitting? How did you handle it? (5 pts)

Yes, this plot does in fact show overfitting. It can be seen that the validation (“val”) loss is continuously increasing while the training loss is continuously decreasing. This heavily indicates that the model is memorizing the training data rather than generalizing to new and unseen data, which is a clear sign of overfitting taking place.

This clear display of overfitting was handled with the implementation of regularization techniques. One regularization technique that was implemented within the code was **EarlyStopping** which works to stop the training once the validation loss was no longer improving to prevent the continuation of overfitting or the transitioning into underfitting.

1. 1.2) The plot obtained after handling any overfitting and underfitting (2.5 pts):



1. Questions on Understanding (20 pts)

For each question, give a brief answer according to your understanding of the problem.

1. 1.3) Question: What is skewness? What problems does it cause? How was it handled in the code? (5 pts)

“Skewness is the degree of asymmetry observed in a set of data. A distribution is right-skewed if the mean is higher than the median, and left-skewed if the mean is below the median.” A few of the problems that it causes are as follows:

- Can lead to models that originally assume normality becoming biased.
- Can reduce the accuracy of the model.
- Skewed features may dominate the learning process.

In the code, skewness was handled by applying a log-transformation to the more positively-skewed features so that the larger values were more compressed and the smaller values were more spread out. This particular handling was done to help make the model as symmetric as possible.

<https://www.investopedia.com/terms/s/skewness.asp>

1. 1.4) Question: How were the missing values in the dataset handled in the code? What other ways can we use? (5 pts)

In the code, the missing values in the dataset were handled by filling them up with the mean of the corresponding feature. The main idea by this specific method was to ensure that the data was filled and complete for training.

A couple of other ways that can be used to handle missing values in a dataset are the following:

- Completely dropping the rows or columns with missing values
- Estimating the missing values by taking the averages of observations from the same class or through the use of similar samples

1. 1.5) Question: What benefit did we get from standardizing the features in the code? (Recall that standardization means subtracting the mean and dividing by the STDEV) (5 pts)

The following are a couple of the main benefits that are typically obtained from standardizing the features of a dataset in the code:

- “Avoids Bias Toward Larger-Scale Features”
- “Improves Model Convergence”

<https://medium.com/@jazeem.lk/why-standardization-is-important-in-machine-learning-9b55a9e03d58>

1. 1.6) Question: What is the purpose of activation functions? Why didn't we add an activation function in the model used in the code? (5 pts)

The purpose of activation functions is to introduce non-linearity into the output of linear functions. Without activation functions, a linear neural network, regardless of how deep, would only be able to perform linear transformations and behave the same as a single-layer linear function. This is because the superposition of a linear function is still a linear function, therefore, by adding non-linearity, activation functions serve the purpose of allowing the network to learn more complex and non-linear relationships within the data, thereby increasing its capacity as well.

An activation function was not added in the code of this model since linear regression was being implemented, meaning that the output was expected to be a continuous and linear value without any transformations. Therefore, the addition of an activation function would have distorted this expected output and would not have preserved the linearity of the function and output.

Part 2: Logistic Regression (45 pts)

1. Experimental Results (45 pts)

Instructions on the Experiments:

- The following table is composed of 5 columns, each with the following definition:
 - **Parameter:** The parameter/stage currently being tuned/changed in the experiments. For example, if it is required to change the data preprocessing, the parameter would be “data preprocessing”; if it is required to change the activation function, the parameter would be “activation function”.
 - **Value:** The current value you tried for this parameter (for example x^2 for artificial features or standardization for data preprocessing). **You are required to try out different values for the parameter in addition to its default value included in the notebook.**
 - **Plot:** the plot you obtained for the model after training at the end of the notebook when you used this specific parameter value.
 - **Accuracy:** The accuracy value indicated by the code cell printing it in the notebook after training the model at the end when you used this specific parameter value.
 - **Overfitting or Underfitting:** Indicate whether the plot shows any overfitting/underfitting when using this parameter value. In case the plot doesn't indicate any overfitting or underfitting, type None.

• Follow the following steps when doing your experiments:

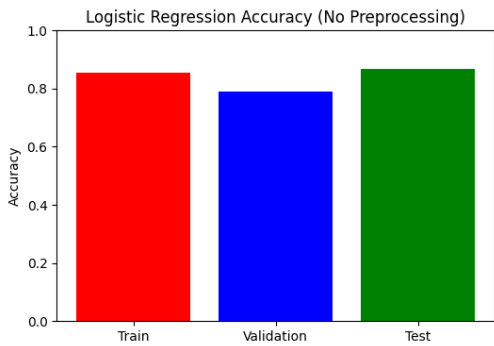
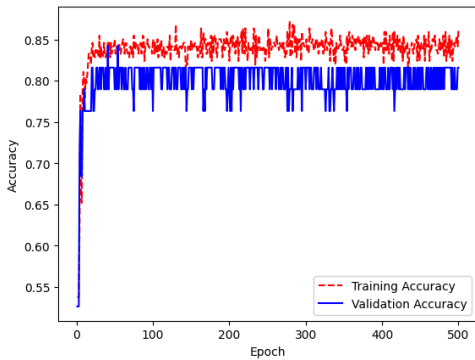
- Complete the code in the notebook and run it using the default parameter values in the notebook to get the results for these default values (plot and accuracy). **Include these results in the rows corresponding to the default parameter values (in all parameters) and indicate any overfitting or underfitting (fill in the TODO cells in these rows).**
- Experiment with each parameter at a time. For example, first try out changing the data preprocessing and reporting the changes in the results. After finishing all required experiments with the data preprocessing, try changing the artificial features and reporting the changes in the results and so on. There is no need to experiment with different combinations of parameters/performing grid search not to waste time (experiment with each parameter on its own). **Although performing grid search is better practice, you are only required to get your hands dirty in the assignment.**

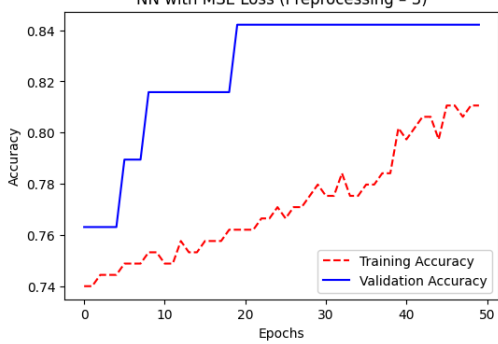
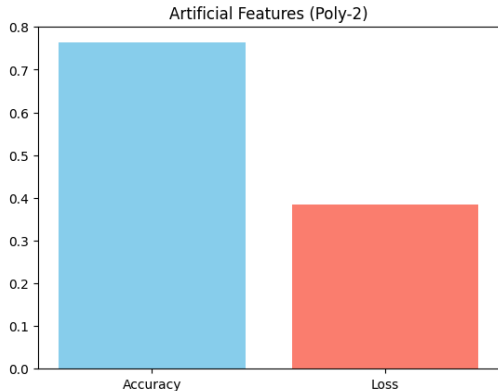
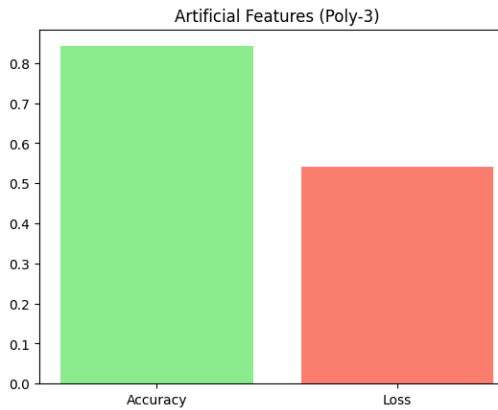
- Each parameter has a number of required experiments indicated by the index after the “-” in the parameter name. Each experiment corresponds to trying out a different parameter

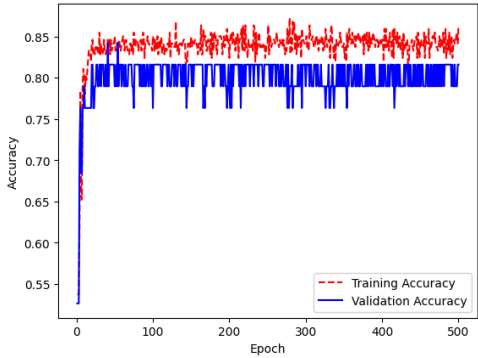
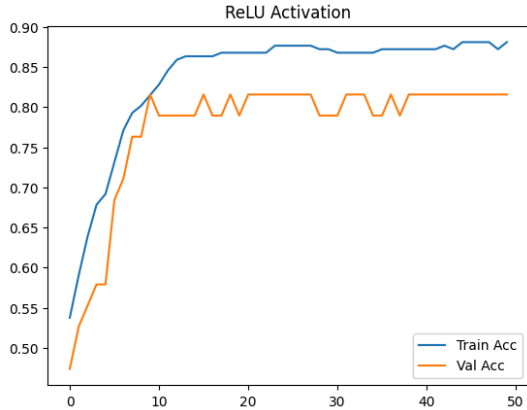
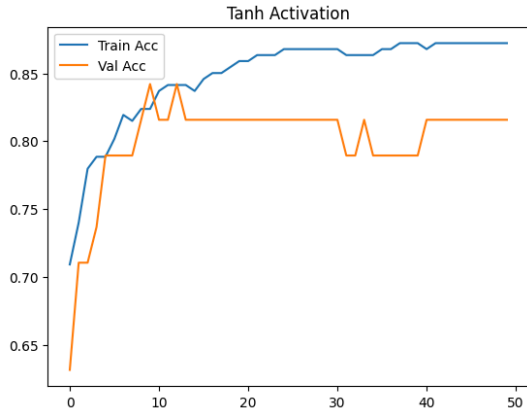
value than those in the experiments before it; where experiment “-1” corresponds to the default parameter value.

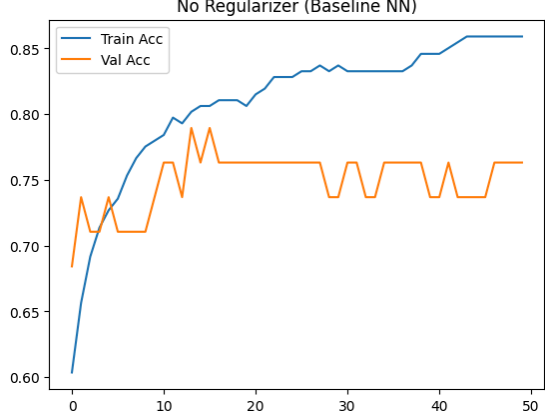
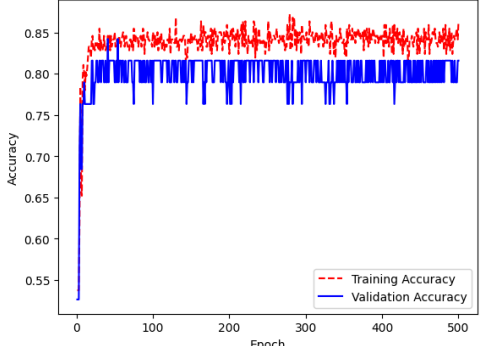
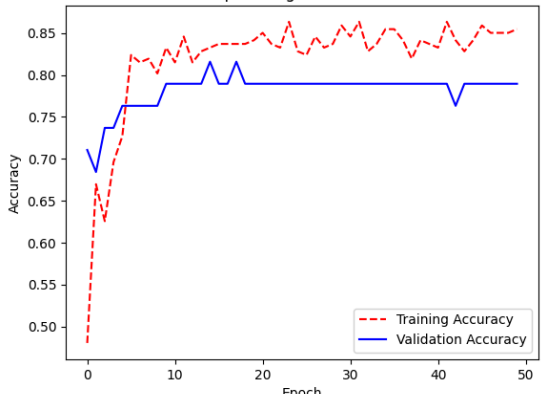
• • **Make sure to perform at least the indicated experiments by filling in all required TODOs in the table to avoid losing grades.**

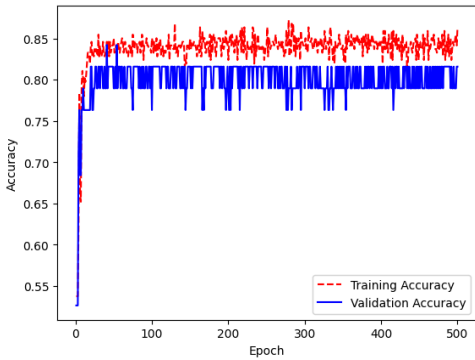
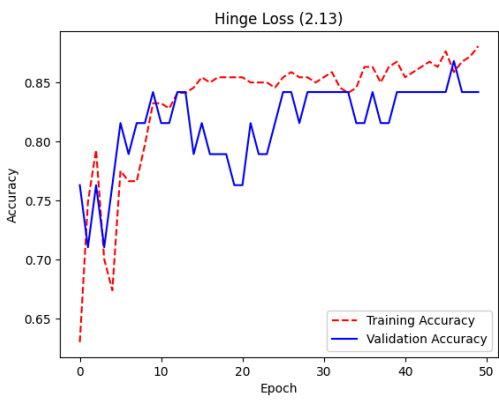
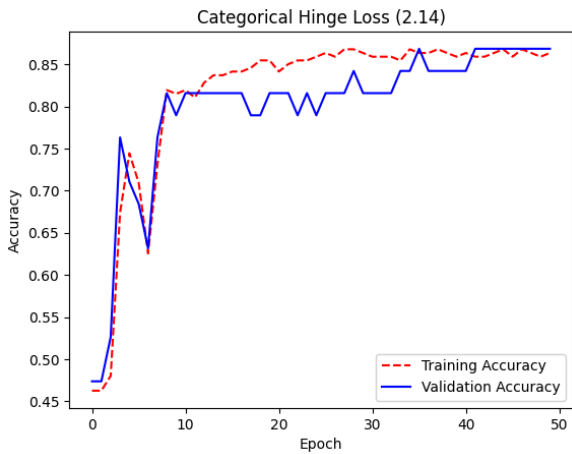
• • There are 10 pts on the default values experiment in total. The remaining experiments are 3.5 pts each.

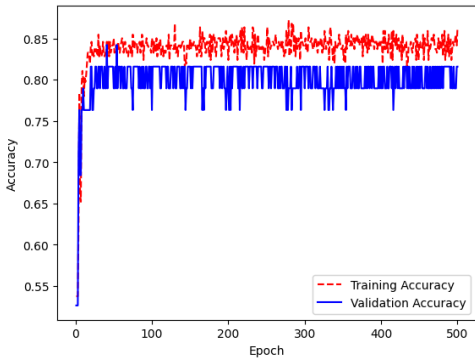
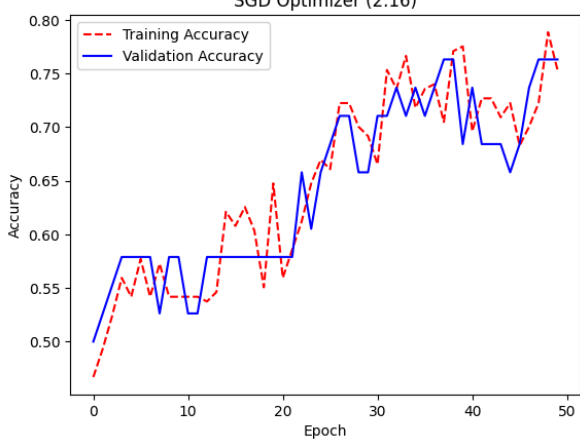
Parameter	Value	Plot	Accuracy	Overfitting or Underfitting?
2.1) Data preprocess - 1	0.384		0.868	Neither
2.2) Data preprocess – 2 (3.5 pts)	0.489		0.816	Neither

2.3) Data preprocess – 3 (3.5 pts)	0.317	 Figure 2.3: NN with MSE Loss (Preprocessing - 3). The graph shows Training Accuracy (red dashed line) and Validation Accuracy (blue solid line) over 50 epochs. Training accuracy starts at 0.74 and increases to 0.81. Validation accuracy starts at 0.76, jumps to 0.84 at epoch 20, and remains constant.	0.737	Overfitting
2.4) Artificial Features - 1	0.385	 Figure 2.4: Artificial Features (Poly-2). The bar chart shows Accuracy (blue bar) and Loss (red bar). Accuracy is approximately 0.76 and Loss is approximately 0.38.	0.763	Underfitting
2.5) Artificial Features – 2 (3.5 pts)	0.540	 Figure 2.5: Artificial Features (Poly-3). The bar chart shows Accuracy (green bar) and Loss (red bar). Accuracy is approximately 0.84 and Loss is approximately 0.54.	0.842	Overfitting

2.6) Activation Function - 1	0.489		0.816	Neither
2.7) Activation Function – 2 (3.5 pts)	0.337		0.895	Neither
2.8) Activation Function – 3 (3.5 pts)	0.358		0.868	Neither

2.9) Regularizer – 1	0.379	No Regularizer (Baseline NN) 	0.842	Overfitting
2.10) Regularizer – 2 (3.5 pts)	0.489		0.816	Neither
2.11) Regularizer – 3 (3.5 pts)	0.377	Dropout Regularizer (2.11) 	0.868	Neither

2.12) Loss - 1	0.489		0.816	Neither
2.13) Loss - 2 (3.5 pts)	0.270		0.895	Neither
2.14) Loss - 3 (3.5 pts)	0.270		0.895	Neither

2.15) Optimizer - 1	0.489	 The graph shows Training Accuracy (red dashed line) and Validation Accuracy (blue solid line) over 500 epochs. The y-axis is labeled 'Accuracy' and ranges from 0.55 to 0.85. The x-axis is labeled 'Epoch' and ranges from 0 to 500. Training accuracy starts at approximately 0.65, rises sharply to about 0.82 by epoch 50, and then fluctuates between 0.82 and 0.85. Validation accuracy starts at approximately 0.55, rises sharply to about 0.78 by epoch 50, and then fluctuates between 0.78 and 0.82.	0.816	Neither
2.16) Optimizer – 2 (3.5 pts)	0.613	 The graph is titled 'SGD Optimizer (2.16)'. It shows Training Accuracy (red dashed line) and Validation Accuracy (blue solid line) over 50 epochs. The y-axis is labeled 'Accuracy' and ranges from 0.50 to 0.80. The x-axis is labeled 'Epoch' and ranges from 0 to 50. Training accuracy starts at approximately 0.45, rises to about 0.55 by epoch 10, and then fluctuates between 0.60 and 0.75. Validation accuracy starts at approximately 0.50, rises to about 0.58 by epoch 10, and then fluctuates between 0.60 and 0.75.	0.763	Underfitting